Lecture 2: Data Structures

Static Variables

- Having name, declaring explicitly within the scope
- Allocated on the **stack** (memory).
- Fixed size (unchange at runtime) leading to suboptimal memory efficiency

```
typedef struct
    {
  char ten[20];
     int    maso;
    }Hocvien;
    Hocvien danhsach[50];
```

    * If the number of students > 50: insufficient capacity (overflow)
    * If the number of students < 50: wasted memory space
    * Faster search via binary search: $O(\log n)$ (if array is sorted).
  - Only use this approach when the exact number of objects is known in advance.

Dynamic Allocation

- Anonymous (unnamed), explicitly allocated at runtime, e.g, `new int(10)`
- Allocated and manually deleted (freed) after use
- Allocated on the **Heap** (memory)
- Unfixed size (High flexibility in size)
- Since it is unamed, how we interact with it?

Pointers

- Poiters is used to store the address of other data structures
- The value of pointer ***p** is the address of the memory space of the variables T type or **nullptr**

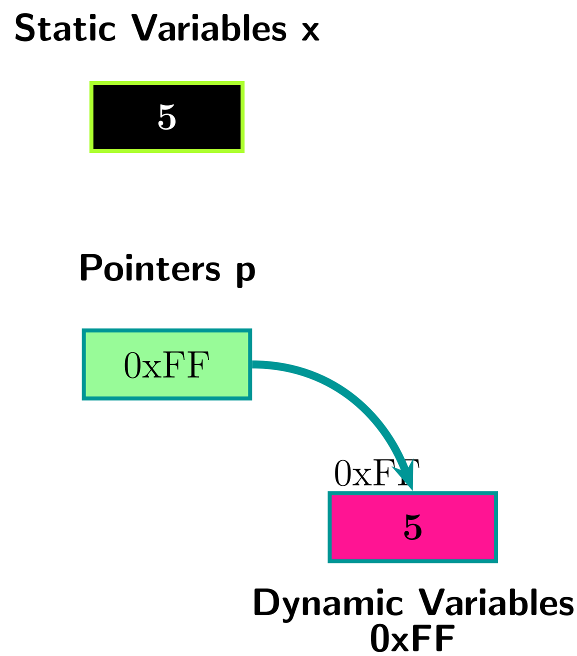
- The pointer is not the dynamic variables
- Use the pointer to store the address and access the dynamic variables through itself

```
int x;
x = 5;


int *p;

p = new(int);

*p = 5;
```



List Type

- a List contains the element has the same data type. List = { data_type }
- List is a linear data structure
- Two basic types:
  - Implicitly linked: Array
  - Explicitly linked: Linked list

Array-based List

- Relationship between item in the array: $[\ldots, a_{i-1}, a_i, a_{i+1}, \ldots]$
- Random Access Memory, Exploit Word-RAM O(1) time random access indexing textbfaddress(i) = address(1) + (i-1)*sizeof(a)
- Pros: Fast direct access t elements by index in
- Cons:
  - Inflexible memory allocation (Fixed size)
  - Set interface (insert_at()..) is not good activation

Linked List

- Data structures for one element:
  - Itself information
  - Address of the next element
- The element is a dynamic variables
- Pros: Flexible memory efficiency